Lecture 1

03 October 2026 00:18

As till now we have made all the chunks, which are nothing but just the small block of data from the Shakespeare text, as we are not going to feed it all at one, we will feed tl
all several chunks in together
It's like not giving a whole homework, but dividing it into small questions, and then giving them

1.C.1 How are inputs given and similarly, Target are got from training any random chunk of data(small part of text)

```
torch.manual_seed(1337)
batch_size = 4
block_size = 8

def get_batch(split):
    data = train_data if split =="train" else val_data
    ix = torch.randint(len(data) - block_size, (batch_size,) )
    x = torch.stack([data[i : i+block_size ] for i in ix ] )
    y = torch.stack([data[i+1 : i+block_size+1] for i in ix ] )
    return x, y

xb, yb = get_batch("train")

print("inputs:")
print(xb.shape)
print(xb)

print("targets:")
print(yb.shape)
print(yb)
```

PART 1:

1.What's torch.manual_seed(1337)

It's just the random numbers produced by the torch, but here it gets lock,
i.e. every time we run the code, the torch will produce the same numbers,
And here the numbers are given for weights

2. What are WEIGHTS?

There are the random values from the torch library, so that the neural network will trained from
That random values

3. Why 1337?

It's just the programmer joke which spells LEET "elite"

PART 2:

1. Block_size and batch_size

Block size will be of 8 only as we are taking only 8 characters and 1 character will be for predictic
Why batch size is 4, cause we are making position tensor of only 4, which will help in making fina
tensor

1. Why is the upper limit len(data) - block_size?

Imagine your dataset has **1,000 characters** in total (indexed from 0 to 999), and your block_size is **8**.
To train the model on a chunk, you need **9 characters total** (block_size + 1): 8 characters for input x, and the 9th character as the target y.

What if we didn't subtract block_size?
If torch.randint randomly picked starting index **997**:

- We need 9 characters starting from index 997: data[997 : 997 + 9] → data[997 : 1006].
- **Problem!** The dataset ends at index 999. There are only 3 characters left (indices 997, 998, 999)!
- Python will give you a broken, incomplete chunk of only 3 characters instead of 8!

The Fix:
By setting the limit to len(data) - block_size ($1000 - 8 = 992$):

- The maximum possible starting index torch.randint can pick is **991**.
- Starting at index 991: data[991 : 991 + 9] gives tokens from index 991 to 999.
- This **guarantees** that every randomly chosen chunk will have enough characters to form a complete block_size chunk without running off the end of the text!

V IMP

2. How for i in ix and torch.stack work step-by-step

Let's look at the exact code line:

```
python
ix = torch.randint(len(data) - block_size, (batch_size,))
x = torch.stack([data[i:i+block_size] for i in ix])
y = torch.stack([data[i+1:i+block_size+1] for i in ix])
```

Let's trace this with an example where batch_size = 4 and block_size = 8.

----------V V IMP----------

Step A: What is ix?
torch.randint picks 4 random starting index numbers from data:

```
python
ix = tensor([100, 500, 25, 800])
```

V IMP : *(Notice ix is NOT indices 0, 1, 2, 3. It contains the 4 actual starting character locations in the dataset!)*

Step B: What does for i in ix do?
In Python list comprehension, for i in ix loops directly over the numbers inside ix:

- **1st loop (i = 100):** grabs data[100 : 108] → Chunk 1 (8 characters)
- **2nd loop (i = 500):** grabs data[500 : 508] → Chunk 2 (8 characters)
- **3rd loop (i = 25):** grabs data[25 : 33] → Chunk 3 (8 characters)
- **4th loop (i = 800):** grabs data[800 : 808] → Chunk 4 (8 characters)

This creates a Python list of 4 separate 1D tensors (each of length 8).

Step C: What does torch.stack(...) do?
torch.stack takes those 4 separate 1D tensors and **stacks them vertically on top of each other** to form a single 2D Tensor (Matrix) of shape (4, 8):

$$Shape: \quad (batch_size, block_size) = (4, 8)$$

text
x = tensor([
[18, 47, 56, 57, 58, 1, 15, 47], # Chunk from index 100
[25, 30, 12, 4, 88, 19, 44, 70], # Chunk from index 500
[1, 11, 9, 33, 2, 55, 8, 14], # Chunk from index 25
[60, 61, 62, 63, 64, 65, 66, 67] # Chunk from index 800
])

Step D: What about y?
y does the exact same loop over ix, but shifted right by 1 index (i+1 : i+block_size+1):

- **1st loop (i = 100):** grabs target data[101 : 109]
- **2nd loop (i = 500):** grabs target data[501 : 509]
- **3rd loop (i = 25):** grabs target data[26 : 34]
- **4th loop (i = 800):** grabs target data[801 : 809]

torch.stack combines them into the target matrix y of shape (4, 8).

Summary Checklist

1. **len(data) - block_size** keeps us from going out-of-bounds at the end of the text.
2. **ix** holds B random starting positions.
3. **for i in ix** loops through each random position to slice a chunk of length block_size.
4. **torch.stack** glue-sticks those 1D chunks into a 2D batch matrix (B, T) for GPU parallel processing.

What is torch.manual_seed(1337)?

In short, **torch.manual_seed(1337) locks PyTorch's random number generator so that it produces the exact same "random" numbers every single time you run your code.**

How it relates to Neural Networks (like your XOR Gate project)

When you built your XOR neural network:

1. You had to **initialize your weights and biases with random numbers** before training.
2. If you didn't fix the random seed, every time you ran your XOR script, Python picked *different* starting weights.
   - On Run 1, maybe it took **500 epochs** to solve XOR.
   - On Run 2, maybe it took **1,200 epochs**.
   - On Run 3, maybe it got stuck in a local minimum!

torch.manual_seed(1337) fixes this problem.
By putting torch.manual_seed(1337) at the top of your script:

- **Initial Weights:** PyTorch will generate the **exact same initial weight values** every time you run the code.
- **Data Sampling:** Any random batch sampling (e.g., torch.randint) will pick the **exact same sequence of numbers**.
- **Results:** Your loss values, training progress, and output tokens will be **100% reproducible**.

1.C.1
```
torch.manual_seed(1337)
batch_size = 4
block_size = 8

def get_batch(split):
    data = train_data if split =="train" else val_data
    ix = torch.randint(len(data) - block_size, (batch_size,) )
    x = torch.stack([data[i : i+block_size ] for i in ix ] )
    y = torch.stack([data[i+1 : i+block_size+1] for i in ix ] )
    return x, y

xb, yb = get_batch("train")

print("inputs:")
print(xb.shape)
print(xb)

print("targets:")
print(yb.shape)
print(yb)

For b in range(batch_size):
  for t in range(block_size):
      context = xb[b, : t+1]
      target = yb[b, t]
      print(f"for the input {context.tolist()} the target is: {target}")
```

V IMP
Let ix = tensor([500, 200, 25, 800])

Output:

here all this chunks 1, 2, 3, 4 are random chunks from the text
Like chunk 1 will be from middle, chunk 2 will be from bottom, etc similarly

```
inputs:
torch.Size([4, 8])
tensor([[24, 43, 58,  5, 57,  1, 46, 43],     ---> chunk 1 of 8 characters NOTE: This Is from random position(ix)  [ 500 : 500+8]
        [44, 53, 56,  1, 58, 46, 39, 58],     ---> chunk 2 of 8 characters NOTE: This Is from random position(ix)  [ 200 : 200+8]
        [52, 58,  1, 58, 46, 39, 58,  1],     ---> chunk 3 of 8 characters NOTE: This Is from random position(ix)  [ 25 : 25+8]
        [25, 17, 27, 10,  0, 21,  1, 54]])    ---> chunk 4 of 8 characters NOTE: This Is from random position(ix)  [ 800 : 800+8]

targets:
```

```
torch.Size([4, 8])
tensor([[43, 58,  5, 57,  1, 46, 43, 39],
        [53, 56,  1, 58, 46, 39, 58,  1],
        [58,  1, 58, 46, 39, 58,  1, 46],
        [17, 27, 10,  0, 21,  1, 54, 39]])

inputs:
torch.Size([4, 8])
tensor([[24, 43, 58,  5, 57,  1, 46, 43],
        [44, 53, 56,  1, 58, 46, 39, 58],
        [52, 58,  1, 58, 46, 39, 58,  1],
        [25, 17, 27, 10,  0, 21,  1, 54]])
targets:
torch.Size([4, 8])
tensor([[43, 58,  5, 57,  1, 46, 43, 39],
        [53, 56,  1, 58, 46, 39, 58,  1],
        [58,  1, 58, 46, 39, 58,  1, 46],
        [17, 27, 10,  0, 21,  1, 54, 39]])

for the input[24] target is: 43
for the input[24, 43] target is: 58
for the input[24, 43, 58] target is: 5
for the input[24, 43, 58, 5] target is: 57
for the input[24, 43, 58, 5, 57] target is: 1
for the input[24, 43, 58, 5, 57, 1] target is: 46
for the input[24, 43, 58, 5, 57, 1, 46] target is: 43
for the input[24, 43, 58, 5, 57, 1, 46, 43] target is: 39
for the input[44] target is: 53
for the input[44, 53] target is: 56
for the input[44, 53, 56] target is: 1
for the input[44, 53, 56, 1] target is: 58
for the input[44, 53, 56, 1, 58] target is: 46
And so on, ……..
```